

Levels and Types of Testing

Рівні та типи тестування програмного забезпечення

Автор: Анна Гуртова (Anna Hurtova)

Курс: Beetroot Academy / Helvetas Ukraine ("Стійкість")

Роль: Trainee QA Manual Engineer

Дата: Червень 2026

Блок 1: Порівняльний аналіз типів тестування

КРИТЕРІЙ	ФУНКЦІОНАЛЬНЕ ТЕСТУВАННЯ	НЕФУНКЦІОНАЛЬНЕ ТЕСТУВАННЯ	ТЕСТУВАННЯ ПОВ'ЯЗАНЕ ЗІ ЗМІНАМИ
Що перевіряється	Технічна реалізація бізнес-функцій системи. Відповідність роботи кнопок, логіки та пошуку вимогам ТЗ.	Характеристики якості роботи системи: швидкість відповіді, надійність, безпека даних, сумісність та локалізація.	Коректність внесення змін у початковий код. Працездатність виправлених ділянок та збереження стабільності старого коду.
Коли проводиться	На початкових етапах розробки, як тільки з'являється перша готова функція програми.	Після того, як базова функціональність системи стала стабільною та пройшла первинні тести.	Щоразу після виходу нового білду коду (оновлення програми), релізу чи виправлення дефектів.
Обмеження	Не відображає поведінку системи при реальних навантаженнях та великій кількості одночасних запитів.	Потребує використання спеціалізованого ПЗ (наприклад, інструменту JMeter).	Потребує значних часових ресурсів на постійне оновлення та запуск тестів при кожній зміні коду.
Особливості	Базується на ТЗ за методом «чорної скриньки». Результати легко оцінити за критерієм «так/ні».	Вимоги часто мають якісний характер, тому потребують фіксації чітких цифрових метрик у специфікації.	Ділиться на два взаємопов'язані види: Ретестінг (перевірка виправлення багу) та Регресія (перевірка, що не зламалося stare).

Різниця між регресією і ретестінгом

Ключовий принцип: Ретестінг перевіряє баг — регресія перевіряє систему.

Ретестінг (Confirmation Testing) — це цілеспрямована перевірка, яка виконується тестувальником виключно для підтвердження того, що конкретний раніше знайдений дефект був успішно виправлений розробником у новій збірці програми.

Натомість **регресійне тестування (Regression Testing)** спрямоване на перевірку вже стабільного, раніше протестованого функціоналу, щоб переконатися, що нові зміни в коді або виправлення багів випадково не зламали сусідні модулі системи.

Головна різниця в об'єктах перевірки полягає в тому, що ретестінг завжди фокусується суворо на **одній конкретній помилці**, тоді як регресія оцінює загальний стан якості **всього програмного продукту** в комплексі.

З погляду тестової документації, ретестінг виконується за тим самим тест-кейсом, який раніше впав, а регресія вимагає запуску величезного набору (Test Suite) різних Smoke або регресійних сценаріїв.

Таким чином, ретестінг відповідає на технічне питання «чи закритий конкретний баг?», а регресійне тестування гарантує бізнесу, що додаток залишається цілісним, працездатним і готовим до фінального релізу.

Блок 2: Чи можливе проведення тільки функціонального тестування?

Відповідь: Ні, проведення лише функціонального тестування є неможливим і призведе до критичних наслідків.

Фінансові втрати

Якщо додаток ідеально виконує функції, але не пройде перевірку продуктивності (навантаження), сервери «впадуть» у перший же вечір від напливу клієнтів. Стартуп втратить гроші на перезапуск інфраструктури.

Юридичні ризики та проблеми з безпекою

Без нефункціональної перевірки безпеки (шифрування даних) персональні дані користувачів та їхні паролі опиняться у відкритому доступі. Це призведе до витоку інформації, судових позовів та повного закриття бізнесу через порушення законів про захист даних.

Повна нерентабельність розробки

Якщо додаток працює без багів, але завантажується довше 3–5 секунд або некоректно відображається на половині сучасних екранів (проблема сумісності), користувачі видалять його за кілька хвилин. Гроші будуть даремно витрачені на команду.

Необхідність проведення димового (Smoke) тестування

Сприйняття необхідності:

Димове тестування — це обов'язковий первинний фільтр якості. Його мета — за 5–10 хвилин перевірити найголовніші функції (чи запускається додаток, чи можна увійти в акаунт), щоб переконатися, що програма взагалі працездатна.

Доречність:

Воно доречне абсолютно завжди, коли QA-інженер отримує від розробників нове оновлення програми. Без димового тесту немає сенсу починати глибоку перевірку: якщо додаток падає прямо при запуску, тестувальник просто зекономить свій час, одразу повернувши білд програмістам на доопрацювання.

Блок 3: Cat Photo App — Нефункціональні вимоги та тест-кейси

Примітка: Повний набір тест-кейсів (позитивні та негативні) задокументований у окремому табличному артефакті — [Тест-кейси та NFR \(Google Sheets\)](#).

Нефункціональні вимоги (NFR) та описи їхніх перевірок

ID	КАТЕГОРІЯ	ВИМОГА	ОПИС ПЕРЕВІРКИ
NFR-01	Продуктивність	Нейромережевий фільтр розпізнавання об'єктів повинен обробляти завантажене фото не довше ніж за 1,5 секунди .	Тестувальник за допомогою вкладки Network в інструментах розробника (DevTools) або внутрішніх логів сервера заміряє час від натискання «Опублікувати» до отримання фінальної відповіді від сервера.
NFR-02	Сумісність	Графічний інтерфейс додатка (кнопки, форми, лічильники) має відображатися без візуальних дефектів на смартфонах з Android 10.0+ та iOS 15.0+ .	QA-інженер запускає додаток на реальних тестових пристроях обох ОС та проводить візуальний контроль елементів на відповідність макетам дизайну.
NFR-03	Безпека	Всі особисті дані профілів та текстові повідомлення в чатах мають передаватися у зашифрованому вигляді за протоколом HTTPS .	Тестувальник запускає проксі-сервер Charles Proxu, перехоплює вихідний трафік із мобільного пристрою та перевіряє, щоб вміст пакетів даних був зашифрованим набором символів, а не відкритим текстом.

ID	КАТЕГОРІЯ	ВИМОГА	ОПИС ПЕРЕВІРКИ
NFR-04	Надійність	Додаток повинен зберігати стабільність роботи і не завершувати сесію аварійно при спробі завантаження файлів не підтримуваного формату (наприклад, .pdf).	Тестувальник навмисно завантажує файл формату .pdf та перевіряє реакцію системи — вона повинна коректно відхилити файл через текстове повідомлення про помилку та продовжити роботу без зависань.